Reown WalletKit & ERC 7811 - Towards a United Onchain UX

March 2025

Luka Isailovic luka@reown.com
Derek Rein derek@reown.com
Jakub Witczak jakub@reown.com

Special thanks to Tony Chen (TokenPocket), Hart Lambur (Across), Vaibhav Chellani (Socket), Eugene Chybisov (li.fi), Josh Rudolf (Ethereum Foundation),, and Pedro Gomes (WalletConnect Foundation) for their feedback and review.

Abstract

Ethereum's primary UX problem, scaling and high fees, was solved through L2s. However, a thriving L2 ecosystem introduced new problems: liquidity fragmentation and interoperability challenges.

Ethereum must provide a compelling end-user experience where end-users can seamlessly use assets from different networks.

ERC-7811 `wallet_getAssets` provides a vendor/wallet agnostic interface to build chain/stablecoin abstracted apps. It however requires wallets to be capable of chain/stablecoin abstraction. WalletKit provides an implementation for wallets and is available in JavaScript/React Native, Kotlin, Swift, and Dart/Flutter.

Suppose a user has a 30USD spending power across USDC/USDT on Arbitrum, Optimism, and Base. They need at least 6 wallet interactions to spend their entire 30USD spending power. This should be just a single wallet interaction.

ERC-7811 `wallet_getAssets`

1. Background/ Motivation

Dapps usually rely on onchain data to determine a user's balance and available assets of the connected accounts. While this approach is usually enough for basic flows, it started to show significant limitations in a multi-chain environment. Although the data exists on each respective chain, apps lack a standardized way to discover which chains to query and which assets are relevant across these chains. Retrieving this distributed information requires multiple separate RPC calls to different networks or reliance on a specific provider to aggregate the data. Wallets inherently possess comprehensive information about a user's assets across all chains, including contextual knowledge about which assets should be considered comparable or equivalent. A standardized interface for communicating this already-aggregated information to apps enables more accurate asset discovery and also opens more opportunities for different solutions like Chain Abstraction.

2. The `wallet_getAssets` method

ERC-7811 introduces `wallet_getAssets`, a JSON-RPC method that enables wallets to declare the full range of assets owned by a user across chains. The method supports filtering by chain, asset type, and specific assets.

The request takes an account parameters and optional filters to control which assets are returned. These filters allow apps to request only specific assets of interest rather than the user's entire portfolio. The response organizes assets by chain ID with each asset including its address, balance, type identifier, and optional metadata.

The specification defines structures for common asset types including native currencies, ERC-20 tokens, and ERC-721 tokens. This standardization ensures consistent representation across different wallet implementations.

```javascript
type Hex = `0x${string}`;
type Address = Hex;
type AssetType = "native" | "erc20" | "erc721" | string;
type Address = Hex;
type AddressOrNative = Address | "native";
type Eip155ChainId = Hex;


type WalletGetAssetsRequest = {
  account: Address;
  assetFilter?: Record<
    Eip155ChainId,
    {
      address: AddressOrNative;
      type: AssetType;
    }[]
  >;
  assetTypeFilter?: AssetType[];
  chainFilter?: Hex[];
};
```

Feature Discovery Wallets can declare support of `wallet_getAssets` either via CAIP-25 `sessionProperties` or via `EIP-5792`'s `capabilities`. Apps can fall back to their old or omnichain implementations if the wallet does not support `wallet_getAssets`.

3. Usage with Wallet Service

The `wallet_getAssets` method presents a unique challenge for certain wallet architectures, particularly mobile wallets that do not share the same execution context as the app.

In these scenarios, direct method invocation would require the user to context switch from the app to the mobile wallet application, creating a fragmented user experience.

To address this limitation, mobile wallet implementations can leverage a dedicated wallet service architecture. This approach enables them to fulfill `wallet_getAssets` requests without requiring direct user interaction with the wallet application for each asset discovery request.

The wallet service acts as an intermediary backend service that maintains information about the user's assets across chains. When a dapp requests asset information via `wallet_getAssets`, the request is routed to the wallet service rather than requiring action in the mobile wallet application itself.

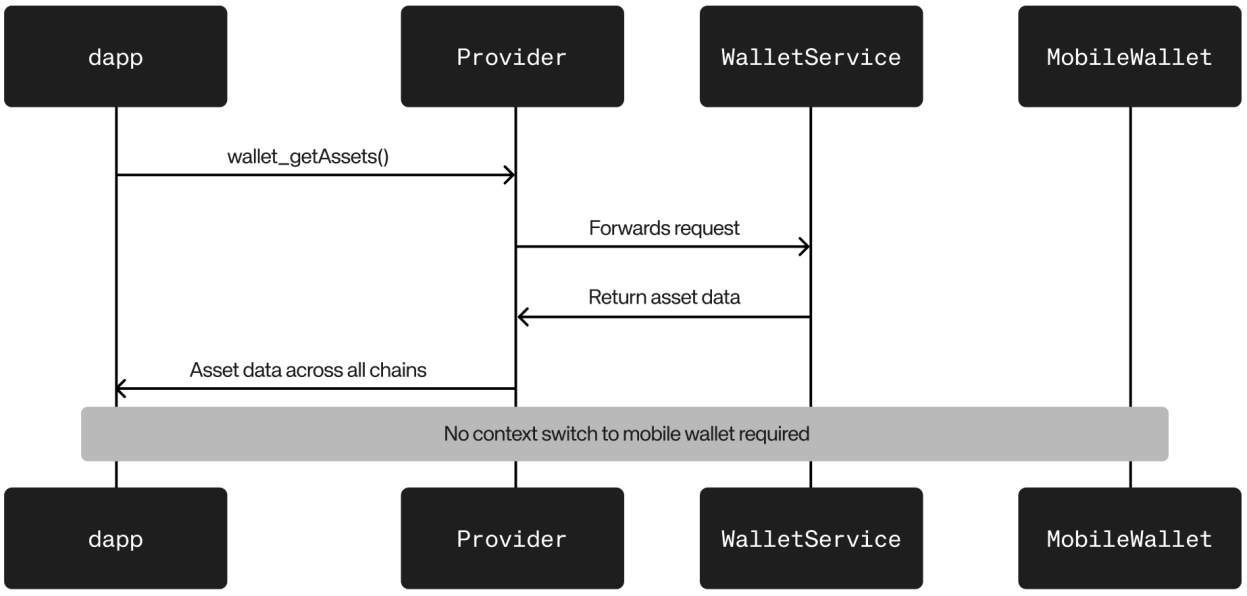


Fig 1: Using 7811 with wallet service

4. Usage for Chain Abstraction use case

ERC-7811 serves as a crucial component for wallet-based chain abstraction solutions like WalletKit. It provides comprehensive asset visibility across chains through a standardized interface that works consistently across different wallet implementations.

The integration of ERC-7811 with chain abstraction solutions addresses one of the most significant UX challenges in the multi-chain ecosystem, the complexity of managing assets across disparate networks. Previously, users needed to manually bridge assets between chains, and track balances across multiple networks. In addition to this, the app would be unaware of this and would prevent the user from starting the flow to preserve the UX on their side.

The flow begins when a dapp requests information about a user's assets via `wallet_getAssets`. This provides the app with a complete view of the user's holdings across all supported chains, enabling it to present options based on the total available liquidity rather than just chain-specific balances. When the user initiates a transaction on a specific chain, the wallet checks if sufficient assets exist on that chain.

If the user has sufficient assets on the target chain, the transaction proceeds normally. However, if the required assets exist on a different chain, the chain abstraction layer within the wallet generates the necessary routing transactions to move assets to the correct chain. These routing transactions, combined with the original transaction, create a unified sequence that appears to the user as a single action.

The power of this integration is its ability to make cross-chain complexity invisible to users. For example, a user with USDC on Optimism can seamlessly interact with a dapp on Arbitrum without manually bridging assets first. The wallet handles all necessary cross-chain movements automatically, presenting a single confirmation screen that encapsulates the entire process.

Chain abstraction leveraging ERC-7811 also creates opportunities for more intelligent transaction routing. The wallet can analyze available assets across chains and determine the most cost-effective or time-efficient path to fulfill the user's request. This might involve selecting specific bridges, utilizing cross-chain liquidity pools, or batching operations to minimize gas costs across multiple networks. Only the resulting total asset allocation per chain will be presented to the app.

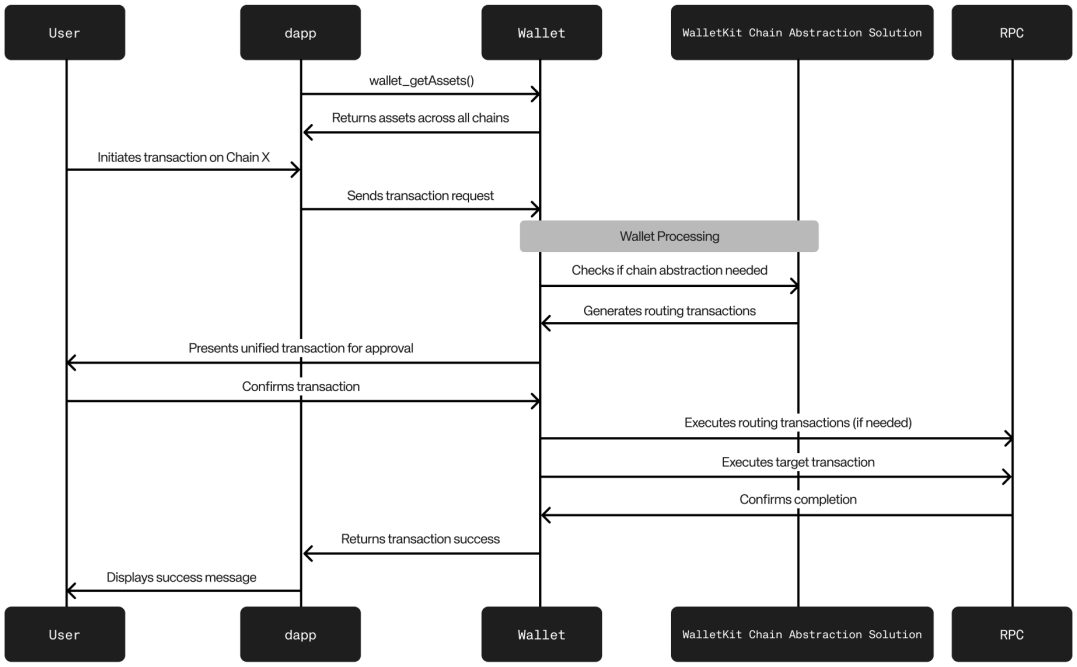


Fig 2: 7811 for Chain Abstraction use case

5. Security and Privacy Considerations

The implementation of ERC-7811 introduces several security and privacy considerations. The method exposes user asset information, requiring appropriate authorization scope. Wallets must ensure that returned data accurately reflects actual holdings to prevent misleading apps or users. Implementations should prevent unauthorized apps from accessing asset information to mitigate phishing risks. To protect user privacy, implementations could return only requested assets rather than full portfolio details.

6. Conclusion

ERC-7811 standardizes cross-chain asset discovery within the EVM ecosystem. By providing a consistent interface for wallets to share asset information with apps, it fills an important gap in the existing infrastructure while enabling new user experiences.

The standard establishes a common language for asset discovery that works across different wallet implementations, chains, and applications. This standardization reduces integration complexity for developers and enables interoperability in the multi-chain ecosystem.

The standard's flexibility, with support for various asset types and filtering mechanisms, ensures it can adapt to ecosystem changes while maintaining compatibility. Its accommodation of different wallet architectures, including wallet services for mobile implementations is making the usage much more streamlined.

As the EVM ecosystem continues to expand across L2s and other scaling solutions, standards like ERC-7811 will be fundamental to creating seamless user experiences. This approach represents a significant step toward a truly interconnected blockchain ecosystem where chain boundaries become invisible to end users.

WalletKit

WalletKit is an SDK embedded into over 600 [1] wallets to enable end-users to connect to external apps. It also ships with a solution designed to solve the fragmentation problem in blockchain ecosystems. Compatible with both Externally Owned Accounts (EOA) and Smart Contract (SC) accounts, it functions seamlessly before and after the Ethereum Pectra upgrade. It is also not constrained to EVM and also supports non-EVM networks e.g. Solana. The concept behind WalletKit is straightforward: by leveraging existing decentralized bridging and swapping infrastructures, it enables users to issue multiple transactions through a single interaction, utilizing the wallet's access to private keys for secure execution.

When an application or a wallet issues a `wallet_sendTransaction` request, WalletKit verifies the availability of sufficient funds across all supported networks. If the user's tokens are present on any supported chain, the transaction can proceed instantly, regardless of which network holds their tokens, creating a unified liquidity experience.

1. Transaction Flow

To facilitate a cross-chain transaction, the wallet follows these steps:

1. Verify if the user's account on the target chain has sufficient funds to complete the transaction.
2. If insufficient, invoke the `WalletKit.prepare()` method to generate the requisite routing transactions for cross-chain asset movement.
3. Sign both the routing transactions and the initial transaction returned by the prepare method.
4. Invoke the `WalletKit.execute()` method to broadcast the routing and initial transactions, awaiting completion.

For example, if a user wishes to send USDC on Arbitrum but holds funds on OP Mainnet, WalletKit prepares and executes the necessary bridging transactions seamlessly, abstracting the complexity of the user.

2. Prepare

The `WalletKit.prepare()` method assesses whether chain abstraction is required. It takes an `InitialTransaction` object—representing the app or wallet-initiated transaction request—and returns a `PrepareSuccess` object.

```
Unset

fun prepare(initialTransaction: InitialTransaction):
PrepareSuccess
```

The return value will be one of the following:

1. `PrepareSuccess.Available`: Returned when chain abstraction is needed, containing routing transactions and funding details.
2. `PrepareSuccess.NotRequired`: Returned when the original chain has sufficient funds, with the original transaction unchanged.
3. `Error:` Returned when insufficient funds exist across all supported chains.

3. Execute

The `WalletKit.execute()` method handles the execution of the chain abstraction flow. It broadcasts the routing and initial transactions in the correct sequence, returning an `ExecuteSuccess` object containing the initial transaction's hash and receipt upon completion.

```
Unset

fun execute(
        prepareAvailable: PrepareSuccess.Available,
        prepareSignedTxs: List<String>,
        initSignedTx: String,
): ExecuteSuccess
```

The method requires:

- `prepareAvailable`:The routing information from `PrepareSuccess.Available`.
- `prepareSignedTxs`: A list of signed routing transactions.
- `initSignedTx`: The signed initial transaction.

The `ExecuteSuccess` object contains:

- Initial transaction hash
- Initial transaction receipt

Supported Tokens

ChainAbstraction currently supports the following stablecoins across these networks, with plans to expand token and network support in future iterations:

Network	Supported tokens
OP Mainnet	USDC, USDT
Base	USDC
Arbitrum	USDC, USDT

4. Sequence diagram

The sequence diagram illustrates the complete WalletKit transaction flow, highlighting two possible entry points (direct wallet usage or dApp request) and two execution paths:

Entry Points

- User Direct Initiation: The user starts a transfer directly in the wallet interface
- dApp Request: External application sends an `eth_sendTransaction` request to the wallet

Flow

1. **Initial Assessment**: Wallet calls `WalletKit.prepare()` to check if the user has sufficient funds on the target chain
2. **Path Determination**: Based on the assessment, the flow branches into two scenarios:

   Chain Abstraction Required:

   - WalletKit returns `PrepareSuccess.Available` with routing information
   - The user signs both bridging transactions and the initial transaction
   - `WalletKit.execute()` manages the sequence of operations:
     - Submits bridging transactions to appropriate protocols
     - Monitors completion of bridge operations
     - Automatically executes the initial transaction once funds arrive
     - Results are returned to the wallet

Chain Abstraction Not Required:

- WalletKit returns `PrepareSuccess.NotRequired`
- Transaction proceeds normally without bridging operations
- Standard transaction flow and confirmation

Completion:

- Results are returned to the originating context (wallet UI or dApp)
- The user receives confirmation of transaction completion

This flow abstracts the complexity of cross-chain operations, presenting users with a seamless experience regardless of where their assets are located within supported networks.

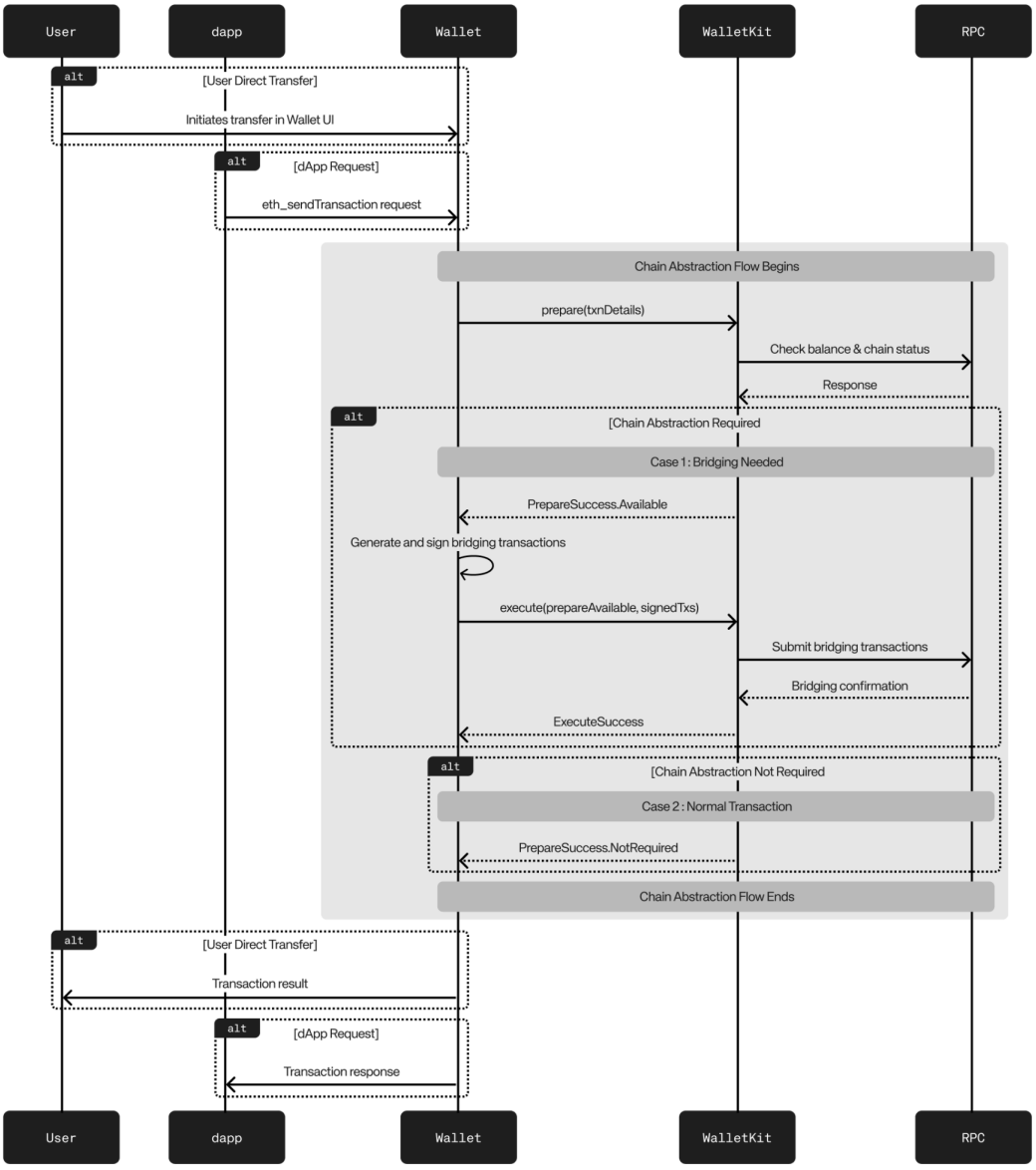


Fig3: WalletKit Transaction Flow

5. Implementation Consideration

Integration Requirements

Wallet developers integrating WalletKit should consider:

1. **User Experience**: Clearly communicate multi-step processes to users
2. **Signature Management**: Implement secure handling of multiple transaction signatures
3. **State Persistence**: Maintain transaction state across potential app restarts

Security Considerations

WalletKit prioritizes security through:

1. **No Private Key Exposure**: All signing operations remain within the wallet
2. **Bridge Selection**: Uses only well-audited, established bridge protocols

Hardware Wallets Considerations

The current WalletKit architecture faces integration limitations when used with hardware wallets. Due to the security architecture of these devices, each transaction requires user authorization through physical interaction with the device.

In cross-chain scenarios requiring multiple transactions, hardware wallet users must provide separate physical confirmations for each step in the sequence. This results in a minimum of three distinct user interactions to complete a flow that software wallets can execute seamlessly with only one click.

6. Conclusion

WalletKit's ChainAbstraction solution streamlines cross-chain transactions, offering users a seamless experience while maintaining compatibility with EOA and SC accounts. By abstracting the complexities of multiple networks into a single interaction, it enhances usability without compromising security, making it a valuable tool for the evolving blockchain ecosystem.

The solution addresses one of the key friction points in web3 user experience - fragmented liquidity across chains - without requiring complex protocol-level changes. This wallet-side approach allows for rapid implementation and iteration, making cross-chain operations accessible to mainstream users.

7. Summary

WalletKit enables a chain/stablecoin abstracted end-user experience for in-wallet apps such as `send`, `swap`, and `stake`. Combined with ERC-7811 this extends to external apps e.g. Uniswap.
This marks an inflection point in onchain UX as there now is a simple solution to the liquidity fragmentation problem that EVM L2s and the success of other L1s e.g. Solana have introduced for the average end-user. The 600+ wallets using WalletKit already and those who don't are strongly encouraged to enable this experience for end-users to experience a frictionless UX.

References

[1] *https://walletguide.walletconnect.network/*